Travel Go

Project Description

TravelGo is a full-stack travel booking web application that allows users to search and book Flights, Trains, Buses, and Hotels from one unified platform — with automated PDF e-ticket generation and email delivery.

Scenario 1: Trip Planning and Destination Recommendation

A user opens the TravelGo platform to plan a vacation. The user enters details such as preferred destination type (beach, mountains, or city), travel dates, and budget. The TravelGo system analyzes the user's preferences and suggests suitable destinations, travel packages, and activities. The platform also displays hotel options, transportation details, and estimated costs. After reviewing the suggestions, the user selects a destination and saves the trip plan for future booking.

Scenario 2: Booking Flights and Hotels

A traveler logs into the TravelGo application and searches for flights and hotels for a specific destination and date. The system retrieves available flight schedules and hotel accommodations from integrated travel APIs. The user compares prices, reviews, and ratings before selecting the preferred options. Once the selection is made, the user proceeds to the payment section and confirms the booking. TravelGo then sends a confirmation email and itinerary details to the user.

Scenario 3: Real-Time Travel Assistance

During a trip, a user accesses the TravelGo mobile app for real-time assistance. The app provides updates on flight status, weather conditions, and nearby attractions. If the user experiences a delay or cancellation, the system suggests alternative flights or accommodations. TravelGo also recommends nearby restaurants, tourist spots, and transportation services based on the user's location, helping the traveler have a smoother and more enjoyable trip.

Technology Stack

Frontend

React — builds the user interface

Vite — runs the React app on a fast dev server

React Router — navigates between pages without reload

Axios — sends API requests to the backend

CSS — styles the app with a dark navy + amber theme

Backend

Python — core programming language

Flask — creates the REST API

Flask-JWT-Extended — handles login tokens (JWT)

bcrypt — securely hashes user passwords

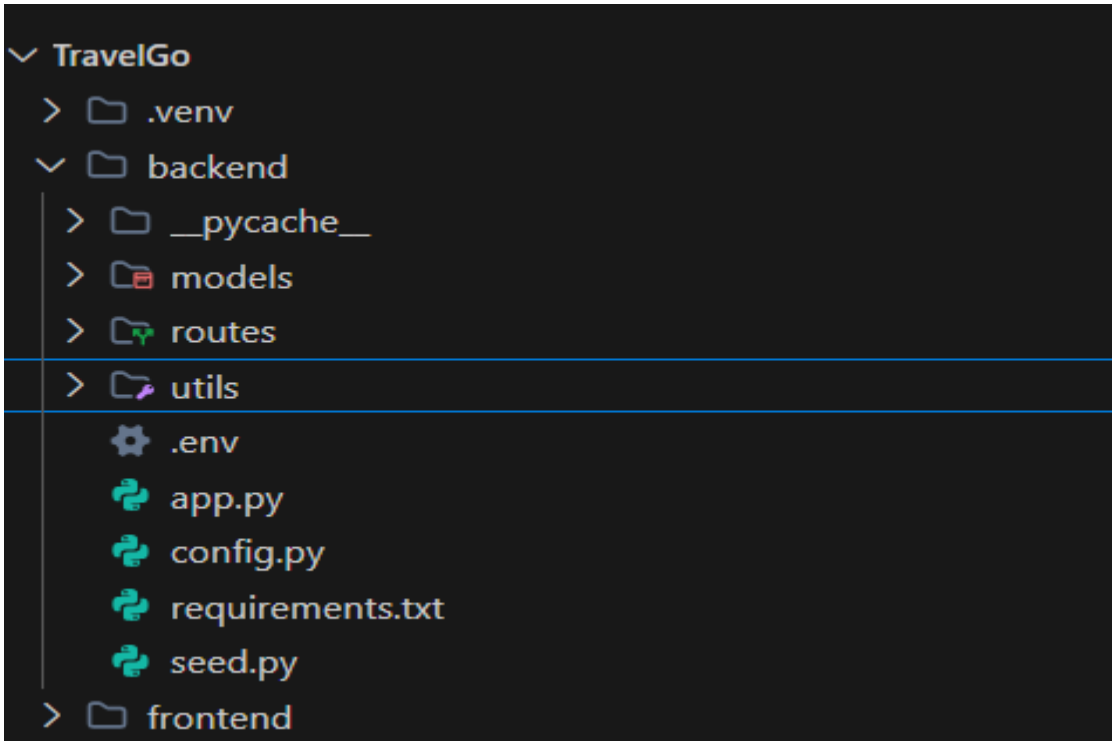
fpdf2 — generates PDF e-tickets

smtplib — sends ticket emails automatically

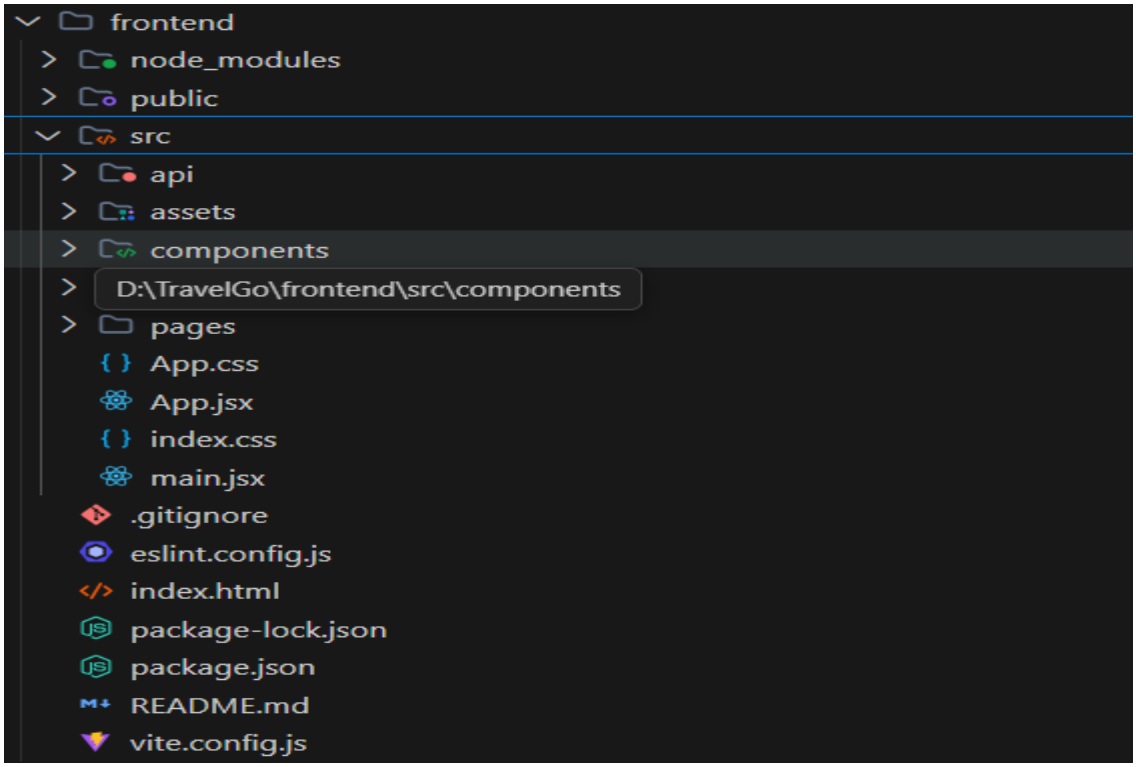
Database

MongoDB — stores users, bookings, flights, trains, buses, hotels

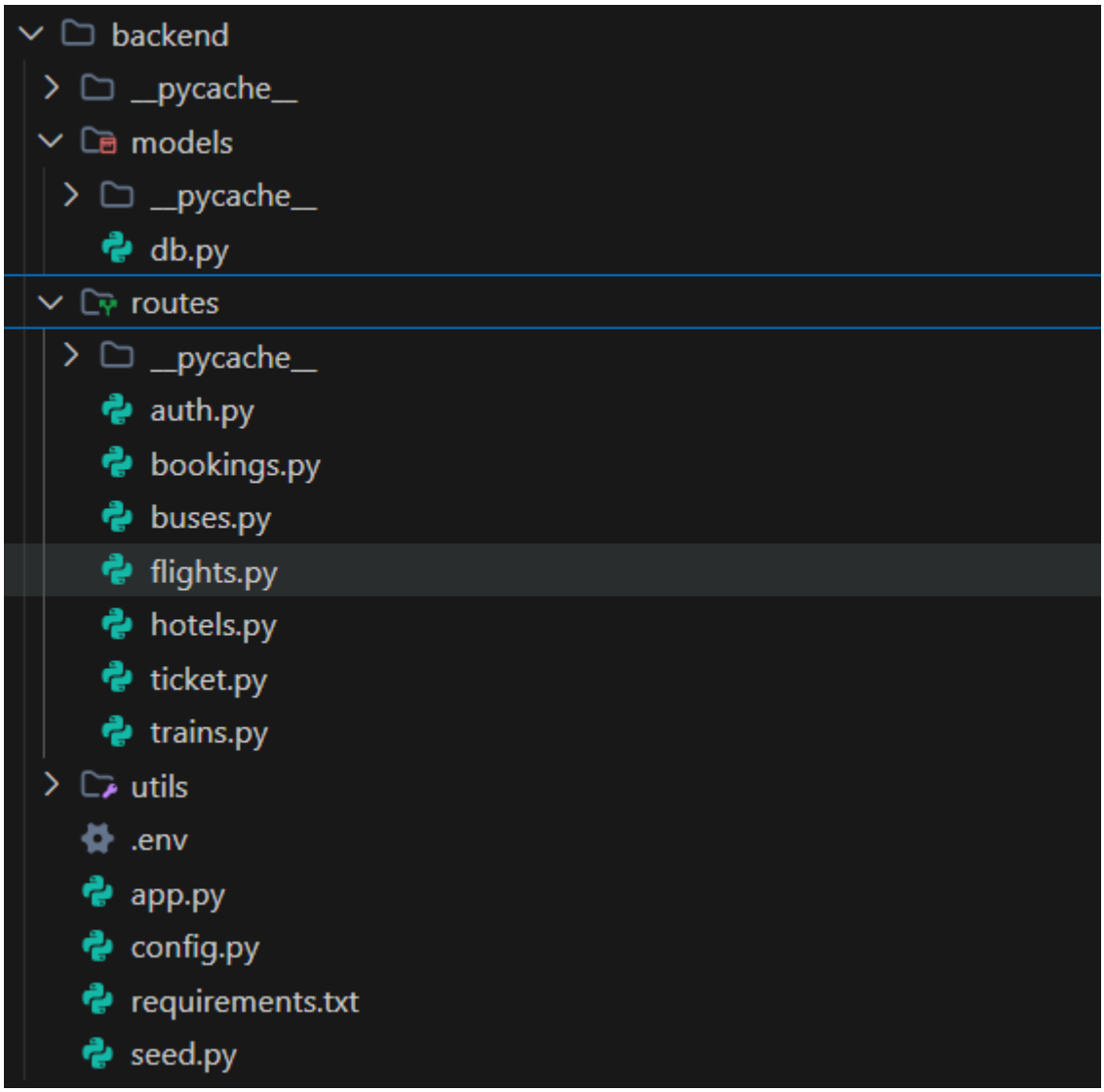
Project Structure



Frontend

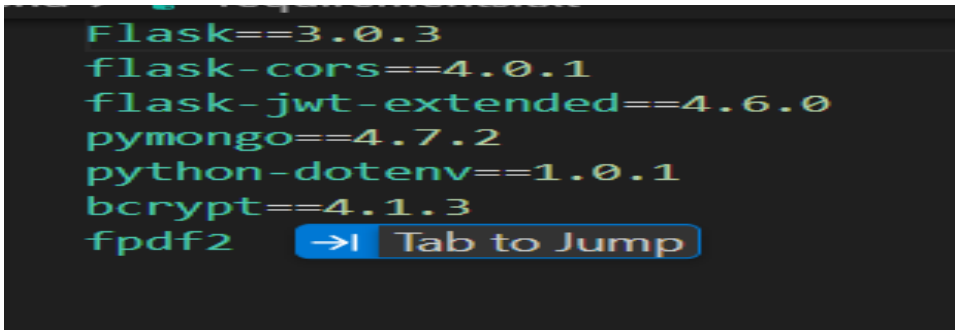


Backend



Installation and Setup

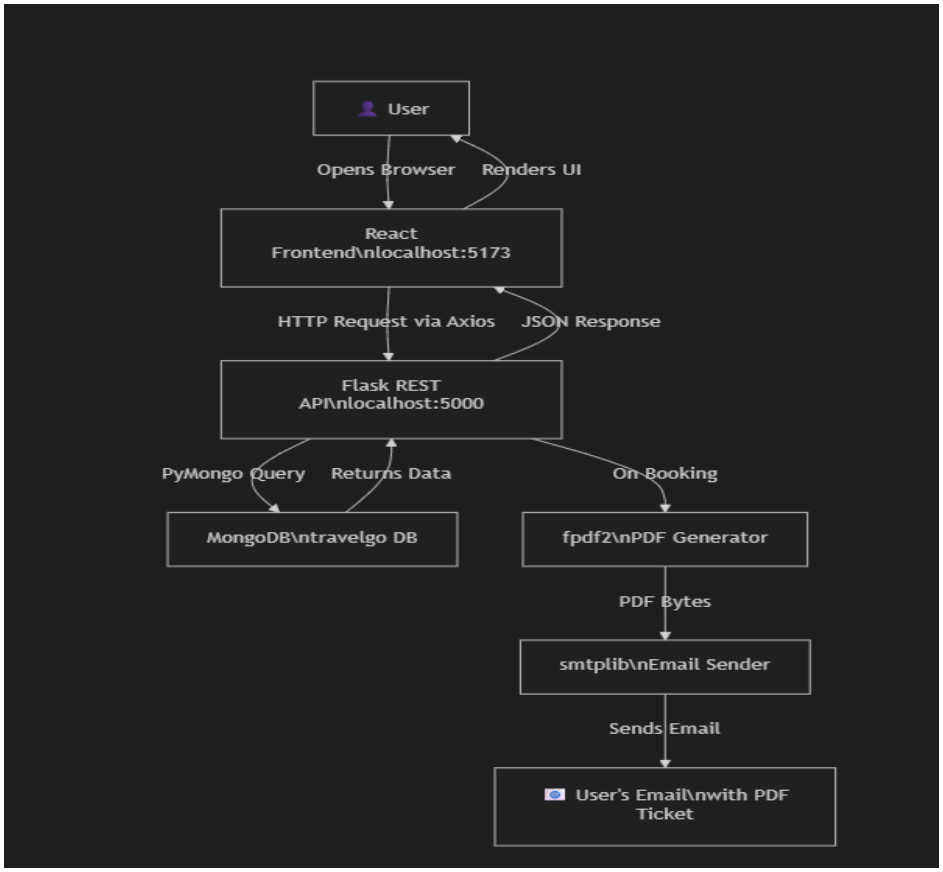
Backend



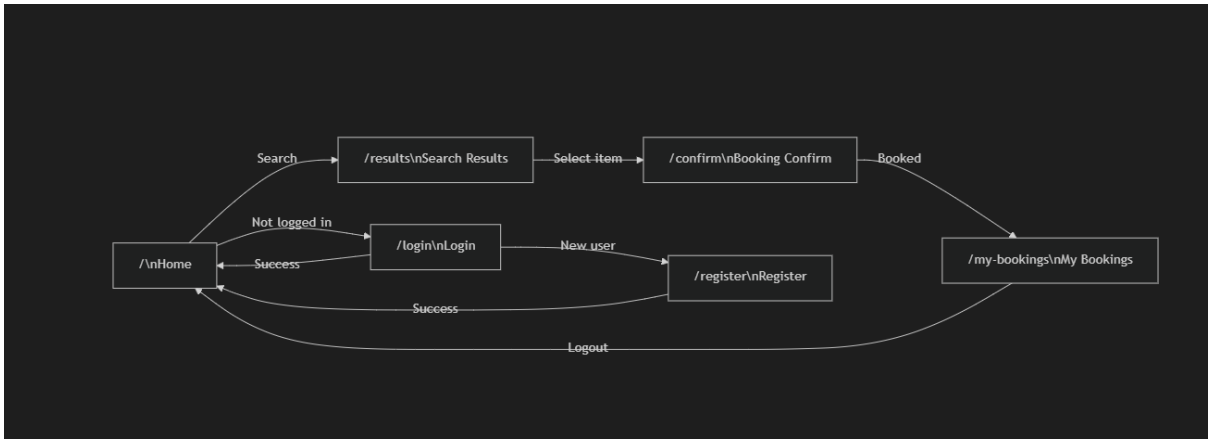
Frontend

```
{
  "name": "frontend",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  ▷Debug
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "axios": "^1.13.5",
    "react": "^19.2.0",
    "react-dom": "^19.2.0",
    "react-router-dom": "^7.13.1"
  },
  "devDependencies": {
    "@eslint/js": "^9.39.1",
    "@types/react": "^19.2.7",
    "@types/react-dom": "^19.2.3",
    "@vitejs/plugin-react": "^5.1.1",
    "eslint": "^9.39.1",
    "eslint-plugin-react-hooks": "^7.0.1",
    "eslint-plugin-react-refresh": "^0.4.24",
    "globals": "^16.5.0",
    "vite": "^7.3.1"
  }
}
```

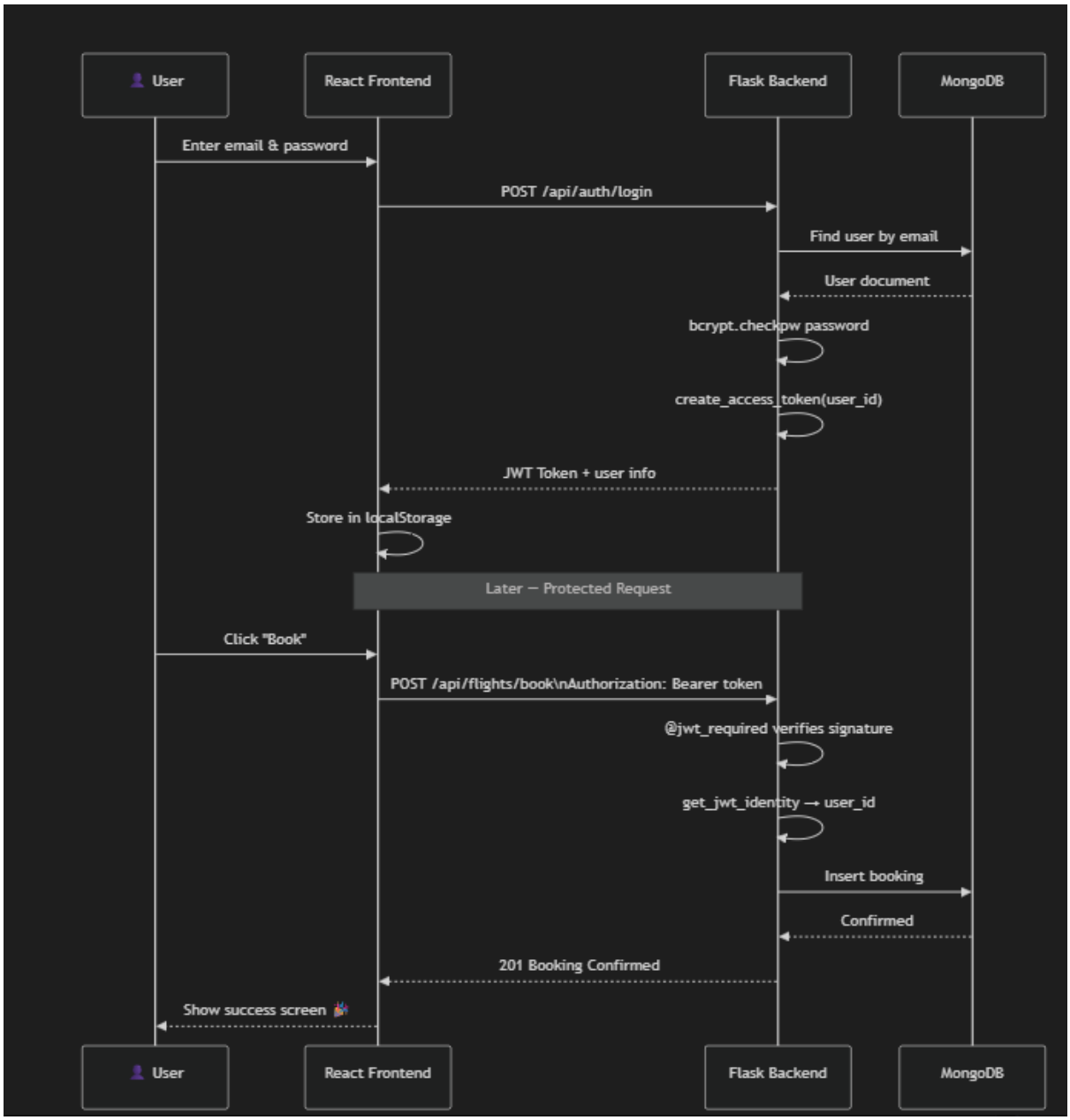
Flow Charts



Frontend Flow

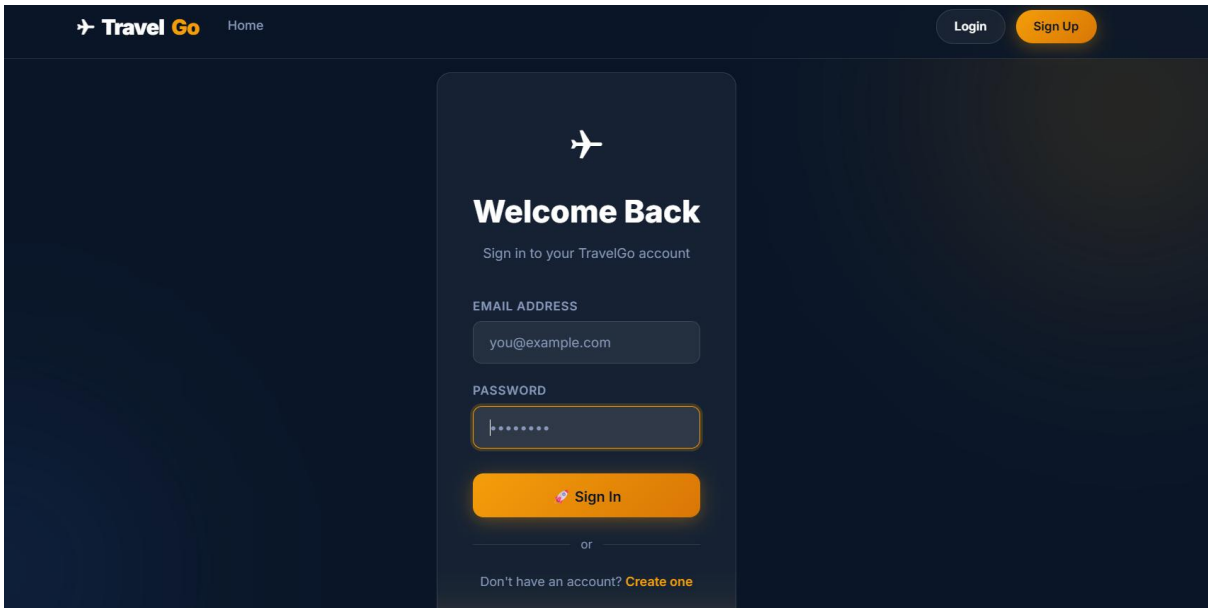


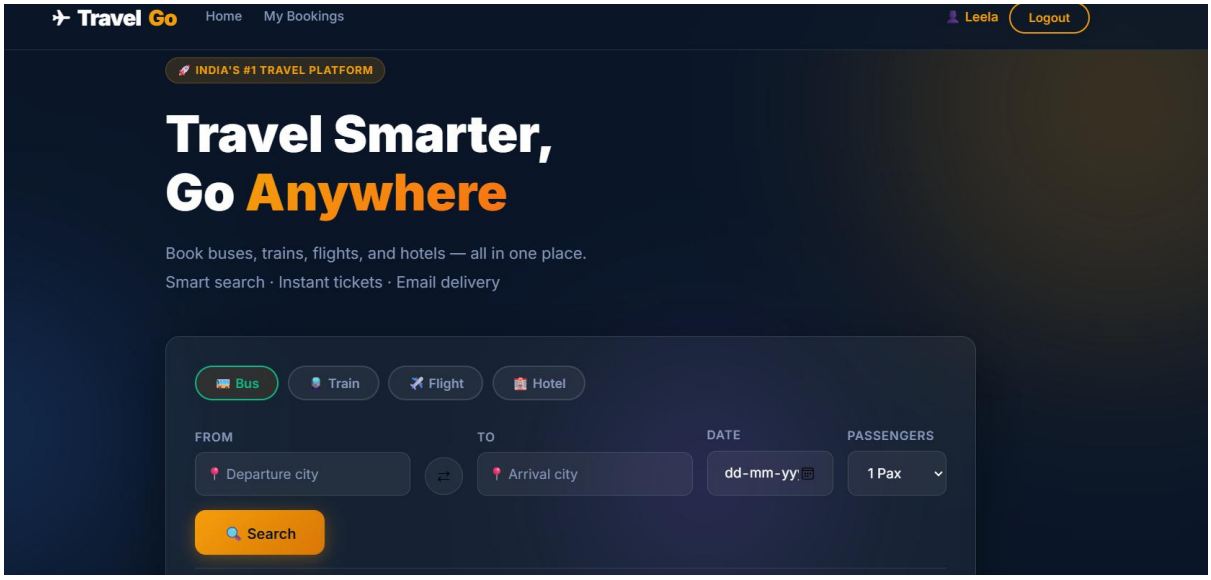
JWT Flow


User
React Frontend
Flask Backend
MongoDB
Enter email & password
POST /api/auth/login
Find user by email
User document
bcrypt.checkpw password
create_access_token(user_id)
JWT Token + user info
Store in localStorage
Later — Protected Request
Click "Book"
POST /api/flights/book\nAuthorization: Bearer token
@jwt_required verifies signature
get_jwt_identity → user_id
Insert booking
Confirmed
201 Booking Confirmed
Show success screen 🎉
User
React Frontend
Flask Backend
MongoDB

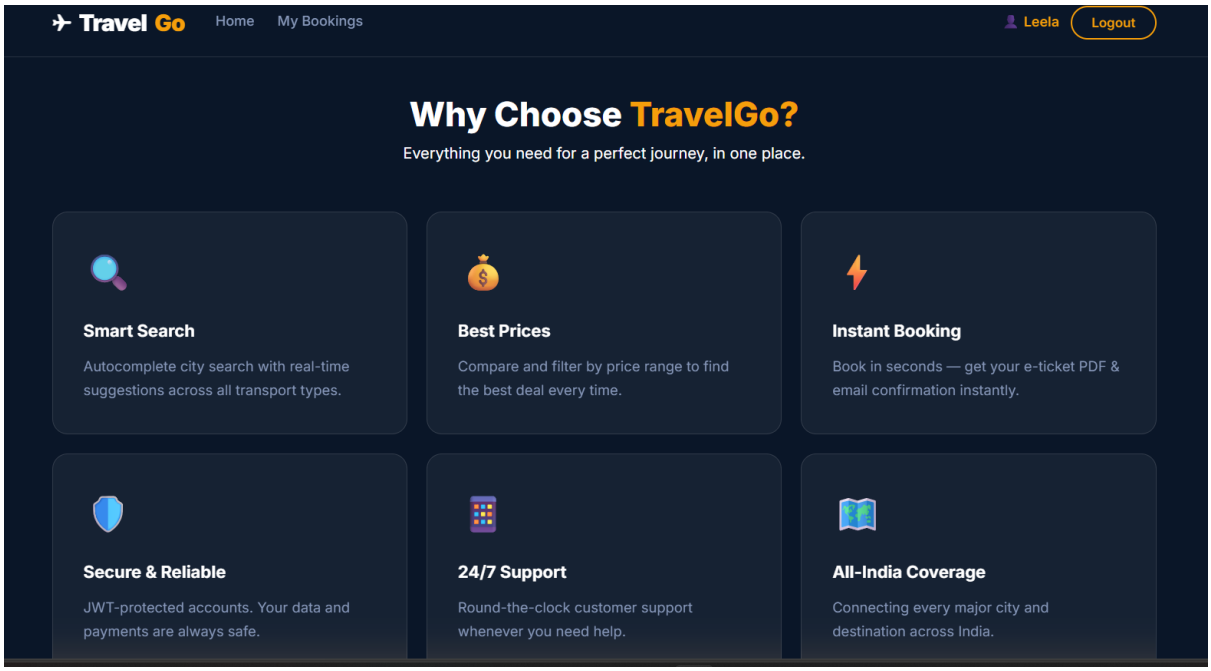
SMARTBRIDGE
Let's Bridge the Gap

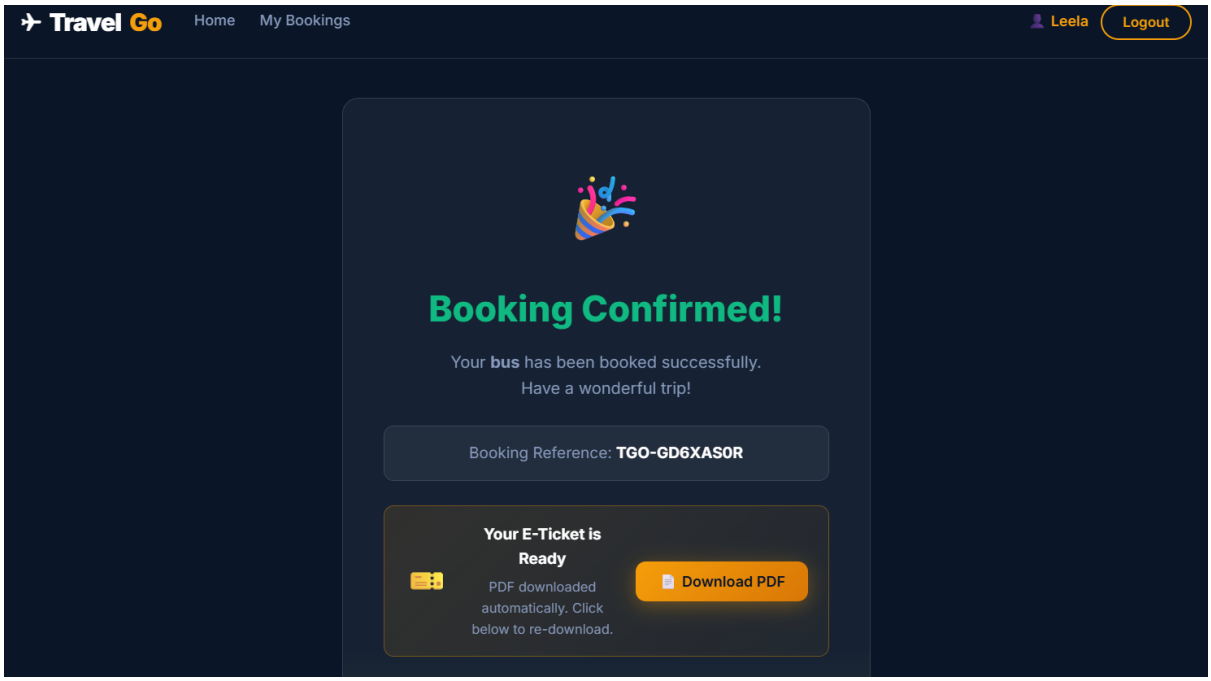
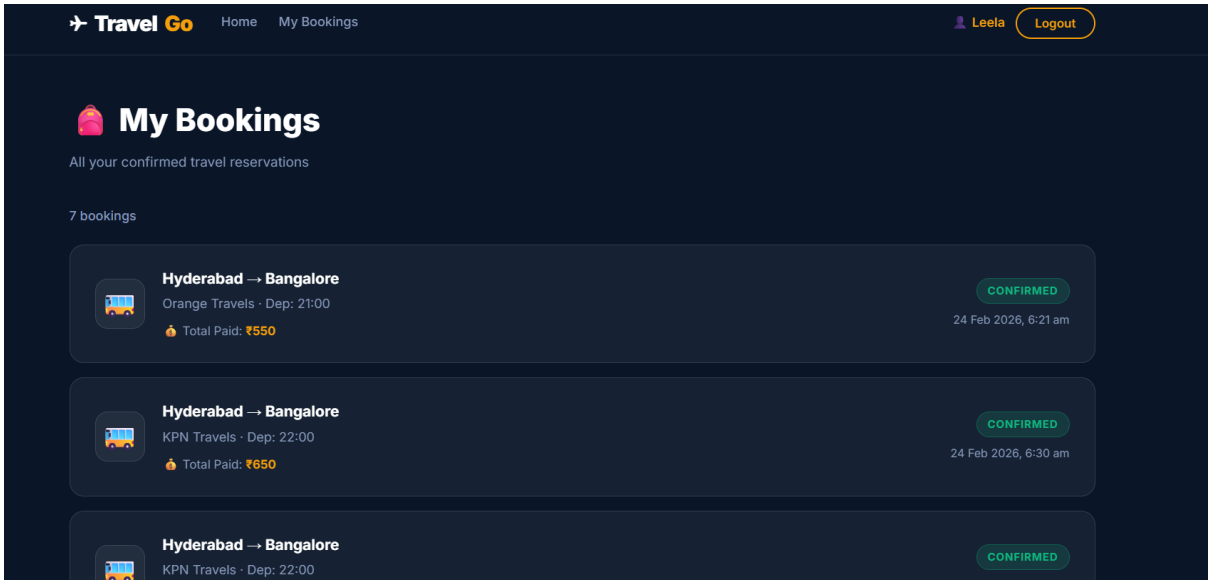
Project Showcase





SMARTBRIDGE
Let's Bridge the Gap





SMARTBRIDGE
Let's Bridge the Gap